

FOR JAVA DEVELOPERS

No approver. \$799.60.

A Spring AI refund agent with no approver: four failures, each measured, and the Java that bounds each one

The model proposes. Java decides.

Sarah Thompson rides with Metro Rides. The trip system marked 40 of her rides as driver no-shows, so each one, on its own, is refundable in full: \$19.99. Policy: no single refund of \$20 or more, \$50 per customer per day, \$100 per merchant per day. Amounts are whole US cents. All data is fictional.

A Spring Boot service exposes two refund tools on an MCP server with Spring AI, and a support agent in Claude Code uses them. Small refunds are paid at once, with nobody approving each one. The authority a person used to hold now lives in Java: the model proposes a refund, the server decides.

At the time of writing, October 2026, the runs used Spring Boot 4.1.1, Spring AI 2.0.1, Java 25 and Postgres 17, with Claude Code 2.1.292 running Claude Sonnet 5.5. Versions change, and a different model may behave differently. The boundaries in the Java are the durable part.

This sheet covers

- Why a per-refund cap is not a limit: forty \$19.99 refunds
- Spending an allowance with one conditional UPDATE, so the check is the write
- Calling the card network after the commit, with a key that cannot pay twice
- A lost answer that keeps the money reserved, and a reconciler that never pays
- A kill switch on the payout path, and an audit row for every decision

Not covered here

- Prompt injection, card data and cross-customer access: the earlier Spring AI support agent repository covers them
- A production deployment: this is a local Claude Code session against a demo server with a demo signing key
- That a model will always stop when refused: one run retried, another stopped, and the money stopped both times

Class paths below are shortened to the part after `src/main/java/com/example/refunds/`. Every block is copied from the repository at commit `33480c4`, and every output from `evidence/`.

01 The four acts, measured

- | | |
|--|---|
| 1. The \$20 loophole, 40 refunds one after another | <code>CapOnlyDesk: 40 paid, \$799.60. ReservingDesk: 2 paid, \$39.98, 38 refused CUSTOMER_DAILY_LIMIT</code> |
| 2. The race, 40 at once, \$100 merchant limit | <code>ReadThenWriteDesk: \$199.90 to \$319.84 over seven runs. ReservingDesk: 5 paid, \$99.95, every run</code> |
| 3. The lost answer, retried once | <code>ReadThenWriteDesk: paid twice, \$39.98. ReservingDesk: UNKNOWN, then EXECUTED, 1 paid, \$19.99</code> |
| 4. The kill switch, 40 proposed | <code>ReservingDesk: 0 paid, \$0.00</code> |

Same customer, same 40 rides of \$19.99, same limits. Only the desk behind the tool changes. Act 2 races 40 requests at once against the \$100 merchant limit only.



02 Everything the agent can touch

Two methods become MCP tools. `list_payments` returns this customer's rides with any refund reason the trip system recorded. `issue_small_refund` takes a payment id, an amount and a reason code, and pays. There is no approval step.

The tool does not take a customer. The customer comes from the signed ticket on every call, so the agent can only reach the person it is helping. Behind the tool sits one interface, `RefundDesk`, with three implementations: two naive desks and one that holds.

RefundTools.java

```
package com.example.refunds;

import java.util.List;
import org.springframework.ai.mcp.annotation.McpTool;
import org.springframework.ai.mcp.annotation.McpToolParam;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.stereotype.Component;

/**
 * The agent's tools. issue_small_refund pays with nobody approving
 * it: the authority a person used to hold now lives in the desk.
 */
@Component
public class RefundTools {

    private final RefundDesk desk;
    private final JdbcClient db;

    public RefundTools(RefundDesk desk, JdbcClient db) {
        this.desk = desk;
        this.db = db;
    }

    public record PaymentLine(
        String id, long amountCents, String eligibleReason) {}

    @McpTool(name = "list_payments",
        description = "This customer's recent payments, with any "
            + "refund reason the trip system recorded.")
    public List<PaymentLine> listPayments() {
        return db.sql("""
            SELECT id, amount_cents, eligible_reason FROM payments
            WHERE customer_id = :c ORDER BY captured_at DESC""")
            .param("c", Ticket.current().customerId())
            .query(PaymentLine.class)
            .list();
    }

    @McpTool(name = "issue_small_refund",
        description = "Refund part or all of one payment. Small "
            + "refunds are paid at once, with no approval.")
    public Decision issueSmallRefund(
        @McpToolParam(description = "Payment id, e.g. RIDE-0412")
        String paymentId,
        @McpToolParam(description = "Amount in US cents")
        long amountCents,
        @McpToolParam(description = "Reason code, e.g. DRIVER_NO_SHOW")
        String reasonCode) {
        return desk.issue(Ticket.current(),
            new RefundProposal(paymentId, amountCents, reasonCode));
    }
}
```

```
}  
}
```

RefundProposal.java

```
package com.example.refunds;  
  
/**  
 * What the agent may put forward. Everything else, the customer,  
 * the merchant, the limits, comes from the server, not from here.  
 */  
public record RefundProposal(  
    String paymentId,  
    long amountCents,  
    String reasonCode) {}
```

RefundDesk.java

```
package com.example.refunds;  
  
/**  
 * Decides a refund the agent proposed, with no person approving it.  
 * Three implementations: two naive ones the video breaks, and the  
 * one that holds.  
 */  
public interface RefundDesk {  
  
    Decision issue(Ticket ticket, RefundProposal proposal);  
}
```

The agent supplies a payment id, an amount and a reason code. The customer comes from the JWT on the MCP call (Ticket.current()).

Read from the code in the repository: RefundTools.java, RefundProposal.java and Ticket.java.



03 Act 1: the \$20 loophole

CapOnlyDesk checks each refund properly: the payment belongs to this customer, the trip system recorded the reason, the refund is not more than the payment, and it is under the cap. Every one of those checks looks at one refund. None of them looks at the day.

The real agent, asked to refund every no-show ride, called `issue_small_refund` forty times. It did nothing wrong, and left alone the three rides with no recorded reason. The card network paid forty times.

CapOnlyDesk.java

```
package com.example.refunds;

import java.util.UUID;

/**
 * Act 1, naive. Each refund is checked on its own: under the cap,
 * eligible, not more than the payment. Nothing adds them up.
 */
public class CapOnlyDesk implements RefundDesk {

    private final Payments payments;
    private final CardNetwork network;
    private final RefundLimits limits;

    public CapOnlyDesk(Payments payments, CardNetwork network,
                       RefundLimits limits) {
        this.payments = payments;
        this.network = network;
        this.limits = limits;
    }

    @Override
    public Decision issue(Ticket ticket, RefundProposal p) {
        var payment = payments.find(ticket, p.paymentId());
        if (payment.isEmpty()) {
            return Decision.refused(p, "UNKNOWN_PAYMENT");
        }
        if (!p.reasonCode().equals(payment.get().eligibleReason())) {
            return Decision.refused(p, "NOT_ELIGIBLE");
        }
        if (p.amountCents() > payment.get().amountCents()) {
            return Decision.refused(p, "MORE_THAN_PAYMENT");
        }
        if (p.amountCents() < limits.maxPerRefundCents()) {
            network.refund(UUID.randomUUID().toString(),
                           p.paymentId(), p.amountCents());
            return Decision.paid(p);
        }
        return Decision.refused(p, "OVER_PER_REFUND_LIMIT");
    }
}
```

●●● The real agent against the cap-only desk, first calls (evidence/agent-cap-only.txt)

Real Claude Code run against the cap-only desk, 2026-10-08. Model claude-sonnet-5-5, Claude Code 2.1.292.

Tool calls (41), in order:

```
list_payments {}
issue_small_refund {"paymentId": "RIDE-0401", "amountCents": 1999, "reasonCode":
"DRIVER_NO_SHOW"}
issue_small_refund {"paymentId": "RIDE-0402", "amountCents": 1999, "reasonCode":
"DRIVER_NO_SHOW"}
```

●●● The same act with no model (evidence/acts.txt)

act 1	cap only	40 refunds of \$19.99	paid 40, \$799.60
-------	----------	-----------------------	-------------------

Claude Code made 41 tool calls: list_payments, then one issue_small_refund per ride, RIDE-0401 to RIDE-0440. Its reply: "Each refund was \$19.99, in full, and each one went through. That comes to \$799.60 in total."

A real Claude Code 2.1.292 session on Claude Sonnet 5.5, 2026-10-08: evidence/agent-*.txt, rerun by scripts/verify.sh --agent. Model behaviour varies run to run.

Cap-only desk, 40 refunds of \$19.99: paid 40, \$799.60.

Measured with no model, 2026-10-08: evidence/acts.txt in the repository, reproduced by scripts/verify.sh or docker compose --profile acts run --rm acts.

04 Write the lead's authority down, and spend it in one statement

RefundLimits is the authority the support lead used to hold: the most for one refund, for one customer in a day, and for one merchant in a day. It is server configuration. The agent cannot read it or change it.

The allowance is not checked and then spent. One UPDATE adds the amount only WHERE reserved_cents + :cents <= limit_cents, and RETURNING hands back what is left. No row matched means it did not fit. Two requests can never both see the same room, and the table's CHECK constraint backs it up.

RefundLimits.java

```
package com.example.refunds;

import org.springframework.boot.context.properties.ConfigurationProperties;

/**
 * The authority a person used to hold, written down. The agent
 * cannot read or change these; they are server configuration.
 */
@ConfigurationProperties("refunds.limits")
public record RefundLimits(
    long maxPerRefundCents,
    long maxCustomerDailyCents,
    long maxMerchantDailyCents,
    String policyVersion) {}
```

src/main/resources/application.properties (excerpt)

```
# The authority a person used to hold. US cents; the business day is
# the database's calendar day (UTC in the demo container).
refunds.limits.max-per-refund-cents=2000
refunds.limits.max-customer-daily-cents=5000
refunds.limits.max-merchant-daily-cents=10000
refunds.limits.policy-version=2026-10-08.1
```

Allowances.java

```
package com.example.refunds;

import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.stereotype.Component;

/**
 * What may still be refunded today. Reserving is one conditional
 * UPDATE: the check and the write are the same statement, so two
 * requests can never both see the same room.
 */
@Component
public class Allowances {

    private final JdbcClient db;

    public Allowances(JdbcClient db) {
        this.db = db;
    }

    /** Returns what is left after reserving, or -1 if it would not fit. */
    public long reserve(String scope, String id, long limit, long cents) {
        db.sql("""
            INSERT INTO allowances
                (scope, scope_id, business_day, limit_cents)
            VALUES (:scope, :id, current_date, :limit)
            ON CONFLICT DO NOTHING""")
            .param("scope", scope).param("id", id)
            .param("limit", limit)
            .update();
        return db.sql("""
            UPDATE allowances
            SET reserved_cents = reserved_cents + :cents
            WHERE scope = :scope AND scope_id = :id
                AND business_day = current_date
                AND reserved_cents + :cents <= limit_cents
            RETURNING limit_cents - reserved_cents""")
            .param("cents", cents)
            .param("scope", scope).param("id", id)
            .query(Long.class)
            .optional()
            .orElse(-1L);
    }

    public void release(String scope, String id, long cents) {
        db.sql("""
            UPDATE allowances
            SET reserved_cents = reserved_cents - :cents
            WHERE scope = :scope AND scope_id = :id
                AND business_day = current_date""")
            .param("cents", cents)
            .param("scope", scope).param("id", id)
            .update();
    }
}
```

```
}  
}
```

src/main/resources/db/migration/V1__schema.sql (excerpt: the allowances table)

```
-- What may still be refunded today, per customer and per merchant. A refund  
-- reserves from both rows in the same transaction, or from neither.  
CREATE TABLE allowances (  
    scope          text    NOT NULL,    -- CUSTOMER or MERCHANT  
    scope_id       text    NOT NULL,  
    business_day   date    NOT NULL,  
    limit_cents    bigint  NOT NULL,  
    reserved_cents bigint  NOT NULL DEFAULT 0,  
    PRIMARY KEY (scope, scope_id, business_day),  
    CHECK (reserved_cents <= limit_cents)  
);
```

Limits in the demo: 2000 cents per refund, 5000 per customer per day, 10000 per merchant per day, policy version 2026-10-08.1.

Server configuration: `src/main/resources/application.properties` in the repository.



05 The desk that holds

In order: the payment's own facts and the kill switch first, and every refusal recorded with its reason. Then one transaction reserves from the customer's and the merchant's allowance and records the refund: both fit, or nothing is reserved. Only after that commits does it call the card network, with the key auto-refund: plus the payment id, the same every time this payment is asked about. A lost answer keeps the reservation.

If the customer reservation fits and the merchant one does not, reserve throws `RefusedInside` and the `TransactionTemplate` rolls the customer reservation back with it.

ReservingDesk.java

```
package com.example.refunds;

import java.util.Optional;
import java.util.UUID;
import org.springframework.dao.DuplicateKeyException;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.transaction.support.TransactionTemplate;

/**
 * The desk that holds. The model proposes; this decides.
 *
 * 1. The payment's own facts and the kill switch come first, and
 *    every refusal is recorded with its reason.
 * 2. One transaction reserves from the customer's and the
 *    merchant's daily allowance, and records the refund. Both fit,
 *    or nothing is reserved.
 * 3. Only after that commits does it call the card network, with a
 *    key that is the same every time this payment is asked about.
 * 4. A lost answer keeps the reservation: the money may have moved,
 *    so the room stays spent until the reconciler knows.
 */
public class ReservingDesk implements RefundDesk {

    private final Payments payments;
    private final Allowances allowances;
    private final CardNetwork network;
    private final KillSwitch killSwitch;
    private final RefundLimits limits;
    private final JdbcClient db;
    private final TransactionTemplate tx;

    public ReservingDesk(Payments payments, Allowances allowances,
                        CardNetwork network, KillSwitch killSwitch,
                        RefundLimits limits, JdbcClient db,
                        TransactionTemplate tx) {
        this.payments = payments;
        this.allowances = allowances;
        this.network = network;
        this.killSwitch = killSwitch;
        this.limits = limits;
        this.db = db;
        this.tx = tx;
    }

    @Override
    public Decision issue(Ticket ticket, RefundProposal p) {
        var found = payments.find(ticket, p.paymentId());
        if (found.isEmpty()) {
            return Decision.refused(p, "UNKNOWN_PAYMENT");
        }
        var payment = found.get();
        if (killSwitch.engaged()) {
```

```
        return refuse(ticket, p, payment, "KILL_SWITCH");
    }
    var problem = checkPayment(p, payment);
    if (problem != null) {
        return refuse(ticket, p, payment, problem);
    }
    String key = "auto-refund:" + p.paymentId();
    var earlier = status(key);
    if (earlier.isPresent()) {
        return answerAgain(p, key, earlier.get());
    }
    Reservation reserved;
    try {
        reserved = tx.execute(s -> reserve(ticket, p, payment, key));
    } catch (RefusedInside rolledBack) {
        return refuse(ticket, p, payment, rolledBack.reason());
    } catch (DuplicateKeyException sameMomentSamePayment) {
        // Another request for this payment won the insert.
        return answerAgain(p, key, status(key).orElse("RESERVED"));
    }
    if (reserved.refusal() != null) {
        return refuse(ticket, p, payment, reserved.refusal());
    }
    return dispatch(p, payment, key);
}

private String checkPayment(RefundProposal p, Payments.Payment pay) {
    if (!p.reasonCode().equals(pay.eligibleReason())) {
        return "NOT_ELIGIBLE";
    }
    if (p.amountCents() <= 0 || p.amountCents() > pay.amountCents()) {
        return "MORE_THAN_PAYMENT";
    }
    if (p.amountCents() >= limits.maxPerRefundCents()) {
        return "OVER_PER_REFUND_LIMIT";
    }
    return null;
}

private record Reservation(String refusal) {}

private Reservation reserve(Ticket ticket, RefundProposal p,
    Payments.Payment pay, String key) {
    long customerLeft = allowances.reserve("CUSTOMER",
        pay.customerId(), limits.maxCustomerDailyCents(),
        p.amountCents());
    if (customerLeft < 0) {
        return new Reservation("CUSTOMER_DAILY_LIMIT");
    }
    long merchantLeft = allowances.reserve("MERCHANT",
        pay.merchantId(), limits.maxMerchantDailyCents(),
        p.amountCents());
    if (merchantLeft < 0) {
        // Undo the customer reservation with the rest of the work.
        throw new RefusedInside("MERCHANT_DAILY_LIMIT");
    }
}
```

```
}
    record(ticket, p, pay, key, "RESERVED", null,
           customerLeft, merchantLeft);
    return new Reservation(null);
}

private Decision dispatch(RefundProposal p, Payments.Payment pay,
                          String key) {
    if (killSwitch.engaged()) {
        release(key, pay, p.amountCents(), "RELEASED");
        return Decision.refused(p, "KILL_SWITCH");
    }
    try {
        network.refund(key, p.paymentId(), p.amountCents());
    } catch (CardNetwork.Timeout lost) {
        setStatus(key, "UNKNOWN");
        return Decision.unknown(p);
    }
    setStatus(key, "EXECUTED");
    return Decision.paid(p);
}

/** The same proposal again, after a timeout or a retry. */
private Decision answerAgain(RefundProposal p, String key,
                             String status) {
    if ("UNKNOWN".equals(status) && network.paid(key)) {
        setStatus(key, "EXECUTED");
        return Decision.paid(p);
    }
    return switch (status) {
        case "EXECUTED" -> Decision.paid(p);
        case "UNKNOWN", "RESERVED" -> Decision.unknown(p);
        default -> Decision.refused(p, "ALREADY_DECIDED");
    };
}

void release(String key, Payments.Payment pay, long cents,
             String status) {
    tx.executeWithoutResult(s -> {
        allowances.release("CUSTOMER", pay.customerId(), cents);
        allowances.release("MERCHANT", pay.merchantId(), cents);
        setStatus(key, status);
    });
}

private Decision refuse(Ticket ticket, RefundProposal p,
                        Payments.Payment pay, String why) {
    if (pay != null) {
        record(ticket, p, pay, "refused:" + UUID.randomUUID(),
              "REFUSED", why, null, null);
    }
    return Decision.refused(p, why);
}

private Optional<String> status(String key) {
```

```
        return db.sql("SELECT status FROM refunds"
            + " WHERE idempotency_key = :k")
            .param("k", key)
            .query(String.class)
            .optional();
    }

    private void setStatus(String key, String status) {
        db.sql("""
            UPDATE refunds SET status = :s, updated_at = now()
            WHERE idempotency_key = :k""")
            .param("s", status).param("k", key)
            .update();
    }

    private void record(Ticket ticket, RefundProposal p,
        Payments.Payment pay, String key, String status,
        String refusal, Long customerLeft,
        Long merchantLeft) {
        db.sql("""
            INSERT INTO refunds (id, idempotency_key, payment_id,
                customer_id, merchant_id, amount_cents, reason_code,
                status, refusal, ticket_agent, policy_version,
                customer_left, merchant_left)
            VALUES (:id, :key, :payment, :customer, :merchant,
                :cents, :reason, :status, :refusal, :agent,
                :policy, :cLeft, :mLeft)""")
            .param("id", UUID.randomUUID())
            .param("key", key)
            .param("payment", p.paymentId())
            .param("customer", pay.customerId())
            .param("merchant", pay.merchantId())
            .param("cents", p.amountCents())
            .param("reason", p.reasonCode())
            .param("status", status)
            .param("refusal", refusal)
            .param("agent", ticket.agent())
            .param("policy", limits.policyVersion())
            .param("cLeft", customerLeft)
            .param("mLeft", merchantLeft)
            .update();
    }

    /** Rolls back the transaction it is thrown in. */
    static class RefusedInside extends RuntimeException {
        final String reason;

        RefusedInside(String reason) {
            super(reason);
            this.reason = reason;
        }
    }
}
```



●●● The same act against the reserving desk (evidence/acts.txt)

act 1 reserving 40 refunds of \$19.99, 38 refused paid 2, \$39.98

The real agent against the reserving desk tried all 40. Two were paid, \$39.98, and 38 came back CUSTOMER_DAILY_LIMIT. Its reply: "I haven't tried to get around that limit." The money stopped at the \$50 allowance because of the desk, not the model.

A real Claude Code 2.1.292 session on Claude Sonnet 5.5, 2026-10-08: evidence/agent-*.txt, rerun by scripts/verify.sh --agent. Model behaviour varies run to run.

Reserving desk, 40 refunds of \$19.99: paid 2, \$39.98, 38 refused.

Measured with no model, 2026-10-08: evidence/acts.txt in the repository, reproduced by scripts/verify.sh or docker compose --profile acts run --rm acts.



06 Act 2: the race

ReadThenWriteDesk does add up the day: it reads what the merchant has refunded today and pays only if this refund still fits under \$100. First a SELECT, then, later, the payment. Between those two lines, other requests are doing the same thing.

Forty refunds fired at once on virtual threads, released by one latch: many read the same total, all see room, all pay. The count changes with timing, which is why a test that passes once proves nothing here. The reserving desk has no read followed by a write, so it was already immune.

ReadThenWriteDesk.java

```
package com.example.refunds;

import org.springframework.jdbc.core.simple.JdbcClient;

/**
 * Acts 2 and 3, naive. Adds up what the merchant refunded today
 * before paying, which stops the salami one request at a time.
 * Two flaws remain: it reads the total, then pays, so parallel
 * requests all see the same room; and a retried request gets a new
 * key, so the network pays it again.
 */
public class ReadThenWriteDesk implements RefundDesk {

    private final Payments payments;
    private final CapOnlyDesk perRefund;
    private final JdbcClient db;
    private final RefundLimits limits;

    public ReadThenWriteDesk(Payments payments, CapOnlyDesk perRefund,
                             JdbcClient db, RefundLimits limits) {
        this.payments = payments;
        this.perRefund = perRefund;
        this.db = db;
        this.limits = limits;
    }

    @Override
    public Decision issue(Ticket ticket, RefundProposal p) {
        var payment = payments.find(ticket, p.paymentId());
        if (payment.isEmpty()) {
            return Decision.refused(p, "UNKNOWN_PAYMENT");
        }
        long spentToday = db.sql("""
            SELECT coalesce(sum(o.amount_cents), 0)
            FROM provider_payouts o
            JOIN payments p ON p.id = o.payment_id
            WHERE p.merchant_id = :merchant
            AND o.paid_at >= current_date""")
            .param("merchant", payment.get().merchantId())
            .query(Long.class)
            .single();
        if (spentToday + p.amountCents()
            > limits.maxMerchantDailyCents()) {
            return Decision.refused(p, "MERCHANT_DAILY_LIMIT");
        }
        return perRefund.issue(ticket, p);
    }
}
```

● ● ● evidence/race-runs.txt

Act 2, the naive read-then-write desk, 40 refunds of \$19.99 fired at once against a \$100.00 merchant

daily limit. Seven runs on 2026-10-08 (six local, one in Docker via compose) (Apple silicon laptop, Postgres 17 in Docker, Java 25):

paid 10, \$199.90

paid 11, \$219.89

paid 12, \$239.88 (twice)

paid 15, \$299.85 (the Docker run)

paid 16, \$319.84 (twice)

The guarded (reserving) desk paid 5, \$99.95 on every run; RefundLimitTests asserts it.

The naive count depends on thread timing, so the video quotes the range, never one run.

● ● ● The recorded acts.txt run (evidence/acts.txt)

act 2 read then write 40 at once, limit \$100.00

paid 11, \$219.89

act 2 reserving 40 at once, limit \$100.00

paid 5, \$99.95

Naive desk, 40 at once against a \$100.00 limit: between \$199.90 and \$319.84 over seven runs. Reserving desk: 5 paid, \$99.95, every run.

Seven runs on 2026-10-08: evidence/race-runs.txt. The naive count depends on thread timing, so only the range is quoted.

readThenWriteOverspendsUnderParallelRequests asserts more than 10000 cents paid;

reservingNeverPassesTheMerchantsDailyAllowance asserts exactly 9995.

Pinned by RefundLimitTests, 9 of 9 passing against Postgres 17 in Testcontainers: evidence/tests.txt, 2026-10-08.

07 Act 3: the refund that paid, but timed out

The card network pays, and the answer never comes back. The caller retries. The naive desk sends the retry with a brand new key, so the network pays again: two payouts for one refund.

The reserving desk reserves and commits before the network call, so the call is never inside the transaction. Its key is stable, so a repeat cannot pay twice. And a lost answer is marked UNKNOWN with the allowance still spent, because the money may already have moved. The retry asks the network what happened to that key and records the payout.

When nobody retries, the Reconciler does the same on a schedule. It never pays: it either records the payout or gives the room back.

ReservingDesk.java (excerpt: dispatch and answerAgain)

```
private Decision dispatch(RefundProposal p, Payments.Payment pay,
                          String key) {
    if (killSwitch.engaged()) {
        release(key, pay, p.amountCents(), "RELEASED");
        return Decision.refused(p, "KILL_SWITCH");
    }
    try {
        network.refund(key, p.paymentId(), p.amountCents());
    } catch (CardNetwork.Timeout lost) {
        setStatus(key, "UNKNOWN");
        return Decision.unknown(p);
    }
    setStatus(key, "EXECUTED");
    return Decision.paid(p);
}

/** The same proposal again, after a timeout or a retry. */
private Decision answerAgain(RefundProposal p, String key,
                             String status) {
    if ("UNKNOWN".equals(status) && network.paid(key)) {
        setStatus(key, "EXECUTED");
        return Decision.paid(p);
    }
    return switch (status) {
        case "EXECUTED" -> Decision.paid(p);
        case "UNKNOWN", "RESERVED" -> Decision.unknown(p);
        default -> Decision.refused(p, "ALREADY_DECIDED");
    };
}
```

Reconciler.java

```
package com.example.refunds;

import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

/**
 * Settles every refund whose answer was lost. It never pays: it asks
 * the network what happened to the key, then either records the
 * payout or gives the room back.
 */
@Component
public class Reconciler {

    private final JdbcClient db;
    private final CardNetwork network;
    private final Allowances allowances;

    public Reconciler(JdbcClient db, CardNetwork network,
                     Allowances allowances) {
        this.db = db;
        this.network = network;
        this.allowances = allowances;
    }

    @Scheduled(fixedDelay = 30_000)
    public int settle() {
        var unknown = db.sql("""
            SELECT idempotency_key, customer_id, merchant_id,
                   amount_cents
            FROM refunds WHERE status = 'UNKNOWN'""")
            .query((rs, n) -> new String[] {
                rs.getString(1), rs.getString(2),
                rs.getString(3), rs.getString(4)})
            .list();
        for (var r : unknown) {
            if (network.paid(r[0])) {
                mark(r[0], "EXECUTED");
            } else {
                long cents = Long.parseLong(r[3]);
                allowances.release("CUSTOMER", r[1], cents);
                allowances.release("MERCHANT", r[2], cents);
                mark(r[0], "RELEASED");
            }
        }
        return unknown.size();
    }

    private void mark(String key, String status) {
        db.sql("""
            UPDATE refunds SET status = :s, updated_at = now()
            WHERE idempotency_key = :k AND status = 'UNKNOWN'""")
    }
}
```

```
        .param("s", status).param("k", key)
        .update();
    }
}
```

● ● ● Answer lost, retried once (evidence/acts.txt)

act 3	read then write	answer lost, retried once	paid 2, \$39.98
act 3	reserving	answer lost: UNKNOWN, retried: EXECUTED	paid 1, \$19.99

Naive desk: paid 2, \$39.98 for one refund. Reserving desk: UNKNOWN, then EXECUTED on the retry, paid 1, \$19.99.

Measured with no model, 2026-10-08: evidence/acts.txt in the repository, reproduced by scripts/verify.sh or docker compose --profile acts run --rm acts.

After a lost answer the customer allowance stays at 1999 cents reserved, and the reconciler records EXECUTED without a second payout.

Pinned by RefundLimitTests, 9 of 9 passing against Postgres 17 in Testcontainers: evidence/tests.txt, 2026-10-08.

08 Act 4: the kill switch stops the money

One row in the database, read on the payout path itself. Taking the tool away from the agent would stop the agent. This stops the money. ReservingDesk reads it before reserving, and again just before the network call, where an already reserved refund is released rather than paid.

In the real run the desk had a \$1,000 allowance, and the switch was engaged in the database the moment the fifth payout landed. Five paid, every one after that refused with the reason recorded, and the agent stopped and asked for escalation. In an earlier run it kept trying. The money stopped both times.

KillSwitch.java

```
package com.example.refunds;

import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.stereotype.Component;

/**
 * One row, checked on the payout path itself. Removing the tool from
 * the agent's prompt would stop the agent; this stops the money.
 */
@Component
public class KillSwitch {

    private final JdbcClient db;

    public KillSwitch(JdbcClient db) {
        this.db = db;
    }

    public boolean engaged() {
        return db.sql("SELECT engaged FROM kill_switch WHERE id = 1")
            .query(Boolean.class)
            .single();
    }

    public void engage(String reason) {
        db.sql("""
            UPDATE kill_switch SET engaged = true, reason = :r
            WHERE id = 1""")
            .param("r", reason)
            .update();
    }
}
```

Kill switch engaged, 40 proposed (evidence/acts.txt)

act 4 reserving

kill switch on, 40 proposed

paid 0, \$0.00



●●● The real run, switch flipped at the fifth payout (evidence/agent-kill-switch.txt, excerpt)

Ledger after the run: payouts 5, total \$99.95. refunds table: EXECUTED 5, REFUSED KILL_SWITCH 5.

An earlier run the same evening (before kill-switch refusals were recorded) attempted all 35 remaining

refunds; this run stopped after 5 refusals. Model behaviour varies run to run; the money stopped both times.

Claude Code made 11 tool calls. Its reply: "I refunded 5 of the 40 driver no-show rides, then the refund system started refusing them. I've stopped." Ledger: 5 payouts, \$99.95.

A real Claude Code 2.1.292 session on Claude Sonnet 5.5, 2026-10-08: evidence/agent-*.txt, rerun by scripts/verify.sh --agent. Model behaviour varies run to run.

09 Every decision is a row

Why was this refund paid? Which payment, which reason code, which policy version, how much of the allowance was left at that moment, and for every refusal, the reason. The model's explanation is not the record. The record is what was proposed, what the policy decided, and what the card network did.

```
src/main/resources/db/migration/V1__schema.sql (excerpt: the refunds table)
```

```
-- Every refund the desk decided on, paid or not, and why.
CREATE TABLE refunds (
  id                uuid PRIMARY KEY,
  idempotency_key   text  NOT NULL UNIQUE,
  payment_id        text  NOT NULL REFERENCES payments (id),
  customer_id       text  NOT NULL,
  merchant_id       text  NOT NULL,
  amount_cents      bigint NOT NULL,
  reason_code       text  NOT NULL,
  -- RESERVED, EXECUTED, UNKNOWN, RELEASED or REFUSED
  status            text  NOT NULL,
  refusal           text,
  ticket_agent      text  NOT NULL,
  policy_version     text  NOT NULL,
  customer_left     bigint,          -- allowance left after this decision
  merchant_left     bigint,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now()
);
```

```
scripts/audit.sh after the guarded run (evidence/audit.txt)
```

payment_id	status	refusal	reason_code	agent	policy	left_cents
RIDE-0401	EXECUTED		DRIVER_NO_SHOW	agent-james	2026-10-08.1	3001
RIDE-0402	EXECUTED		DRIVER_NO_SHOW	agent-james	2026-10-08.1	1002
RIDE-0403	REFUSED	CUSTOMER_DAILY_LIMIT	DRIVER_NO_SHOW	agent-james	2026-10-08.1	
RIDE-0404	REFUSED	CUSTOMER_DAILY_LIMIT	DRIVER_NO_SHOW	agent-james	2026-10-08.1	
RIDE-0405	REFUSED	CUSTOMER_DAILY_LIMIT	DRIVER_NO_SHOW	agent-james	2026-10-08.1	

(5 rows)

RIDE-0401 left 3001 cents of the customer allowance, RIDE-0402 left 1002, and RIDE-0403 onwards were refused CUSTOMER_DAILY_LIMIT under policy 2026-10-08.1.

Measured with no model, 2026-10-08: evidence/acts.txt in the repository, reproduced by scripts/verify.sh or docker compose --profile acts run --rm acts.

10 Nine tests, no model

Every act is proved without a model, against a real Postgres in a container: forty small refunds capped at the customer's allowance, forty at once never past the merchant's, a lost answer paid once, and the kill switch paying nothing. They give the same answer every time, which the model never will.

```
● ● ● evidence/tests.txt
```

```
ok  the kill switch stops the money not just the agent
ok  read then write pays twice when the answer is lost
ok  reserving pays once and keeps the room spent
ok  reserving stops at the customers daily allowance
ok  read then write overspends under parallel requests
ok  each refund still needs its own reason
ok  cap only pays forty small refunds
ok  the reconciler records a lost answer without paying
ok  reserving never passes the merchants daily allowance
Tests run: 9, Failures: 0, Errors: 0 (5.0 s)
```

Tests run: 9, Failures: 0, Errors: 0.

Pinned by RefundLimitTests, 9 of 9 passing against Postgres 17 in Testcontainers: evidence/tests.txt, 2026-10-08.



Before an agent refunds with no approver

1. Limit the total per customer and per merchant per day, not only the size of one refund
2. Keep the limits in server configuration the agent can neither read nor change
3. Take the customer from the signed ticket, never from a tool argument
4. Take eligibility from the system of record, never from the model's justification
5. Spend the allowance with one conditional UPDATE, so the check and the write are the same statement
6. Reserve every allowance and record the refund in one transaction, or reserve nothing
7. Call the payment provider after the commit, never inside the transaction
8. Send a key that is the same every time this payment is asked about
9. On a lost answer, keep the allowance reserved and reconcile by asking the provider; never pay from the reconciler
10. Read the kill switch on the payout path, not in the prompt or the tool list
11. Record every decision, paid or refused, with the policy version and the allowance left

Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- The repository: the three desks, the tests, the evidence and scripts/verify.sh**
<https://github.com/code-with-sam-dev/spring-ai-refund-limits>
- The earlier Spring AI support agent, where a person approved every refund**
<https://github.com/code-with-sam-dev/spring-ai-support-agent>
- Spring AI reference documentation**
<https://docs.spring.io/spring-ai/reference/>
- PostgreSQL 17: UPDATE, including RETURNING**
<https://www.postgresql.org/docs/17/sql-update.html>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Claude and Claude Code are trademarks of Anthropic. Spring is a trademark of Broadcom. Java is a trademark of Oracle. PostgreSQL is a trademark of the PostgreSQL Community Association of Canada. This is an independent, unofficial build produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-10-08 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.